

Logic Programming

Rodica Potolea
Camelia Lemnaru
Richard Ardelean

Lecture #6

Incomplete structures

Agenda

- **Incomplete structures**
 - Meaning & utility
 - Lists
 - Trees

Incomplete structures

Lists

- Incomplete structures:
 - Structures ending in variables
 - = instead of the specific empty structure it ends in a variable.
- This **IS** a list ending in a variable: $L_1 = [1, 2, 3 | \mathbf{A}]$
 - Tail is a VARIABLE
 - Can you specify its length?
 - Tail can be anything (including a list, such as $[4, 5]$). BUT in this case, it unifies with the variable, and the structure would be a homogeneous one: $L_1 = [1, 2, 3, \mathbf{4}, \mathbf{5}]$
- This **IS NOT** a list ending in a variable $L_2 = [1, 2, 3, \mathbf{B}]$
 - Only the last element is a variable, which can be anything (including a list, such as $[4, 5]$). BUT in this case, it unifies the variable, the structure would be a heterogeneous one:
 $L_2 = [1, 2, 3, \mathbf{[4, 5]}]$

Incomplete Lists - search

```
member(X, [X|_] ) .  
member(X, [_|T] ) :-  
    member(X, T) .
```

```
[q1] ?- L=[1,2,3|_] , member(2,L) .  
true, L=[1,2,3|_]
```

```
[q2] ?- L=[1,2,3|_] , member(4,L) .  
true, L=[1,2,3,4|_]
```

- Wrong behavior. Why? How can we fix the issue?

Incomplete Lists - search

```
member_IL(X, [X|_]) :- !.           % item found, stop. For good!
member_IL(X, [_|T]) :-             % item not found,
    member_IL(X, T).                % go search in tail
member_IL(_, L) :-
    var(L), !,                      % reached final free variable
    fail.                           % stop with failure.
```

```
[q1] ?- L=[1,2,3|_], member_IL(2, L).
true, L=[1,2,3|_]
```

```
[q2] ?- L=[1,2,3|_], member_IL(4, L).
true, L=[1,2,3,4|_]
```

- Again, same wrong behavior. How come we didn't fix the issue?

Incomplete Lists - search

```
member_IL(_, L) :-  
    var(L), !,                % this order: check, cut, fail  
    fail.  
member_IL(X, [X|_]) :- !.    % cut mandatory  
member_IL(X, [_|T]) :-  
    member_IL(X, T).
```

```
[q1] ?- L=[1,2,3|_], member_IL(2, L).  
true, L=[1,2,3|_].           % exe stops on clause 2
```

```
[q2] ?- L=[1,2,3|_], member_IL(4, L).  
false.                       % exe stops on clause 1
```

- It NEVER reaches clause #3 in the previous code (previous slide) ? Why?
- **Stop conditions** for **incomplete** structures are (i) with **cut** (!) and (ii) **ALWAYS come first**. Otherwise, it is as if they are missing!
 - Starts with failure condition(s). It MUST be explicit (NEVER default).
 - Continues with the successful stop condition(s).

Incomplete Lists – insert

```
insert_IL(X,L):-  
    var(L),                % reached the final free variable?  
    !,                    % stop backtracking  
    L=[X|_].              % update the list with the new added item  
insert_IL(X,[X|_]):-!.    % item found, and don't allow backtracking  
insert_IL(X,[_|T]):-  
    insert_IL(X,T).
```

```
[q1]?- L=[1,2,3|_], insert_IL(4,L).  
true, L=[1,2,3,4|_].
```

```
[q2]?- L=[1,2,3|_], insert_IL(3,L).  
true, L=[1,2,3|_].
```

- q1: pure insert behavior (stop on clause 1)
- q2: member behavior (stop on clause 2)

Incomplete Lists – insert

- We don't really need clause 1. Clause 2 covers it (default).

```
insert_IL(H, [H|_]) :- !.  
insert_IL(H, [_|T]) :-  
    insert_IL(H, T).
```

```
[q1]?- L=[1,2,3|_], insert_IL(4,L).  
true, L=[1,2,3,4|_].
```

Has pure insert behavior. Clause 1 behaves as:

```
insert_IL(X,L):-  
    var(L), !, L=[X|_].
```

```
[q2]?- L=[1,2,3|_], insert_IL(3,L).  
true, L=[1,2,3|_].
```

Has member behavior. Clause 1 behaves as:

```
insert_IL(X, [H|_]) :- X=H, !. % member(X,[X|_]):- !.
```

Incomplete structures - review

- Incomplete structures (lists, trees)
 - Replace the empty structure at the end **with a variable**
- Used as I/O structure (saves time and space)
- Change the behavior of regular predicates
- *Failure* conditions NEED to be:
 - ALL Explicit (no default failure in the absence of the clause)
 - Come FIRST (otherwise, is as if it does not exist)
- *Success* conditions should be:
 - All explicit (as before)
 - Start with the condition for the variable at the end
 - which comes right after the failure condition(s)

Search in BST

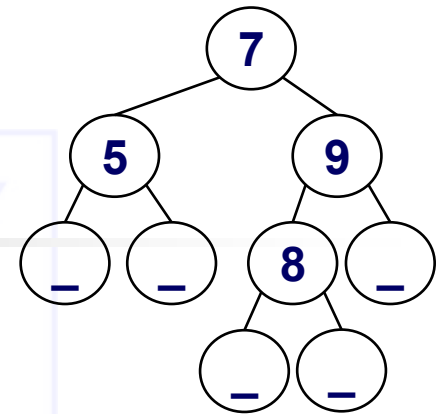
- This is an Incomplete (ending in variable) BST

```
tree(t(7, t(5,_,_),t(9,t(8,_,_),_))).
```

```
search(Key, t(Key,_,_)):-!,  
    write("found").
```

```
search(Key, t(K,L,_)):-  
    Key<K,!,  
    search(Key, L).
```

```
search(Key, t(_,_,R)):-  
    search(Key, R).
```



- The search predicate works well for a **regular** BST
- What behavior would it have in case of an incomplete tree?

Search in BST

- It has **dual** significance in the search process:
 - if **K present**, reports so
 - else **inserts** it as leaf

```
tree(t(7, t(5,_,_),t(9,t(8,_,_),_))).
```

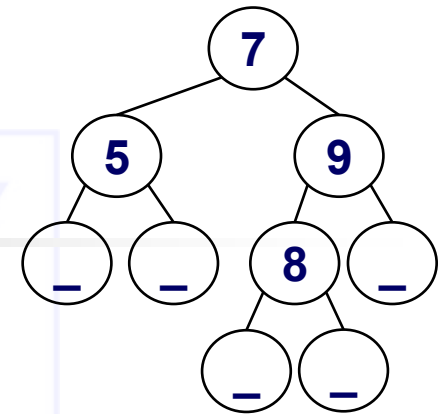
```
search_ins(Key,t(Key,_,_)):-!,
    write("found").
```

```
search_ins(Key, t(K,L,_)):-
    Key<K,!,
    search_ins(Key, L).
```

```
search_ins(K, t(_,_,R)):-
    search_ins(Key, R).
```

```
[q]?-tree(T), search_ins(9,T).
    % behaves as search (= search_ins)
```

```
[q]?-tree(T), search_ins(6,T).
    % behaves as insert (= search_ins)
```



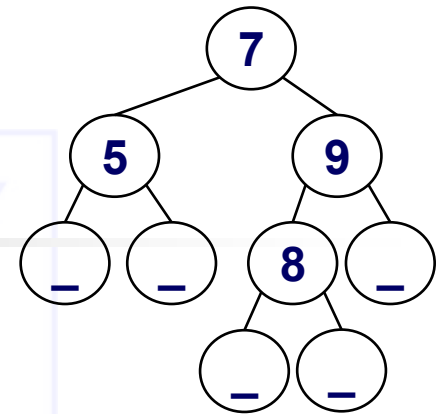
Search in BST

```
[q] ?-tree(T), search_ins(9, T).  
    % behaves as search (= search_ins)
```

```
search_ins(Key, t(Key, _, _)) :- !,  
    write("found OR inserted").
```

The meaning of the clause is:

```
search_ins(Key, T) :-  
    nonvar(T), T = t(K, _, _), Key = K, !,  
    write("found OR inserted").
```



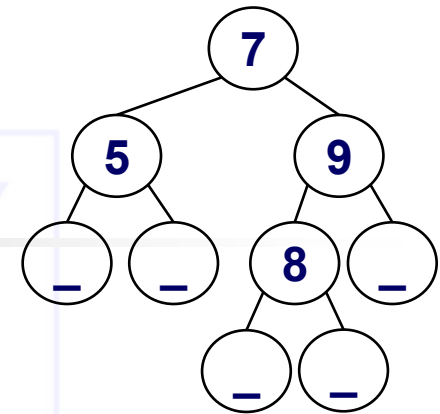
Search in BST

```
[q] ?-tree(T), search_ins(6, T).  
    % behaves as search (= search_ins)
```

```
search_ins(Key, t(Key, _, _)) :- !,  
    write("found OR inserted").
```

The meaning of the clause is:

```
search_ins(Key, T) :-  
    var(T), !, T=t(Key, _, _),  
    write("found OR inserted").
```



PURE Search in BST (search by Key)

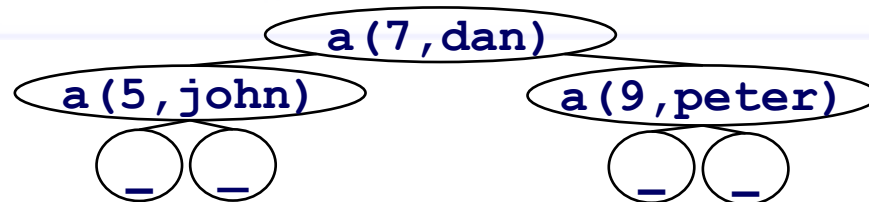
```
search_IT(Key, T) :-  
    var(T), !,  
    fail.  
search_IT(Key, t(Key, _, _)) :-  
    !,  
    write("found").  
search_IT(Key, t(K, L, _)) :-  
    Key < K, !,  
    search_IT(Key, L).  
search_IT(Key, t(_, _, R)) :-  
    search_IT(Key, R).
```

Computer Science

Search in BST (search based on Key)

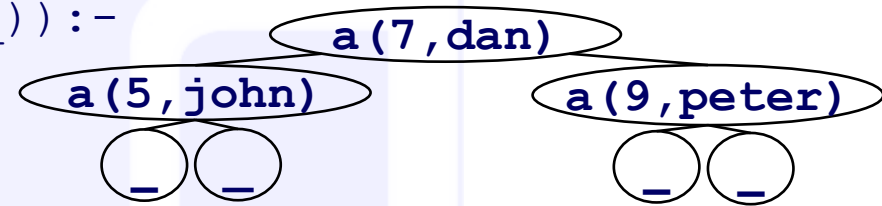
```
search_ins(Key, Info, n(_, a(Key, Info), _)) :- !.
search_ins(Key, Info, n(L, a(K, _), _)) :-
    Key < K, !,
    search_ins(Key, Info, L).
search_ins(Key, Info, n(_, _, R)) :-
    search_ins(Key, Info, R).
```

```
[q] ?- tree(T), search_ins(9, I, T).
true, I=peter
```

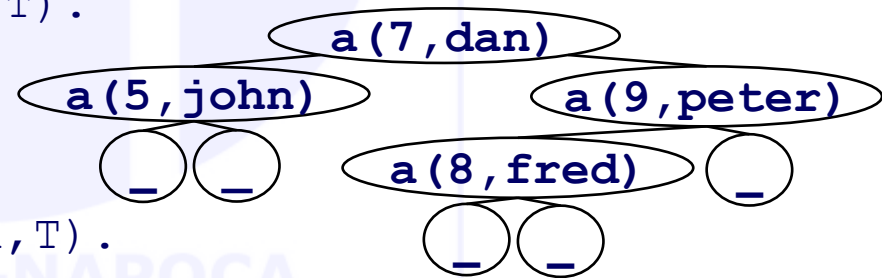


Search in BST (search based on Key)

```
search_ins(Key, Info, n(_, a(Key, Info), _)):-!.
search_ins(Key, Info, n(L, a(K, _), _)):-
    Key<K, !,
    search_ins(Key, Info, L).
search_ins(Key, Info, n(_, _, R)):-
    search_ins(Key, Info, R).
```

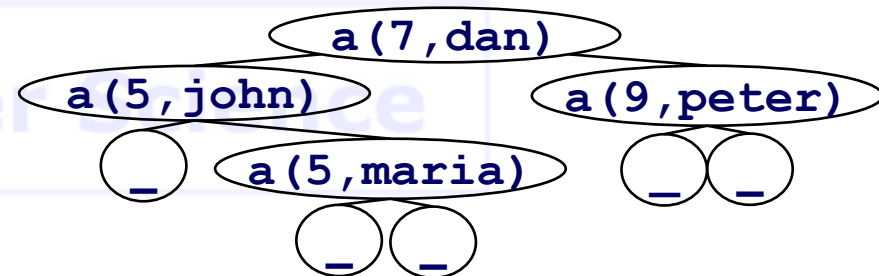


```
[q] ?- tree(T), search_ins(8, fred, T).
true, ... % T increases
```



```
[q] ?- tree(T), search_ins(5, maria, T).
true, ... % T increases
```

- Does it fail?
 - If Yes, How?
 - If Not, Why? How is the tree affected?



Search/insert in Incomplete BST

- The regular search in a BST in an Incomplete BST has dual behavior:
 - Returns found - in the case it is in tree
 - Inserts it at the bottom level - in the case it is not found
- If you want to fail when the key is not found?
 - Add an explicit **first** clause as:

```
search(_, T) :-
    var(T), !,
    fail.
```

- What other situations are there?
 - Nodes contain <key,value>
 - search to fail if key exists even with a different value (info) !

```
search(Key, Info, n(_, a(K, I), _)) :-
    Key=K, !,
    Info/=I, !,
    fail.
```